



MANUAL TÉCNICO

Instalación, mantenimiento y calibración

Gas Station Automation

Versión	1.0.0
Elaborado por	Frank Vargas

Guía para mantener el proyecto y calibrar cada pantalla

Contenido

Esta guía define un flujo seguro para mantener el proyecto actualizado, trabajar en ramas personales y calibrar coordenadas según cada pantalla.

- **1. Objetivo y regla principal**
- **2. Requisitos previos**
- **3. Clonar el proyecto por primera vez**
- **4. Mantener main actualizado antes de trabajar**
- **5. Crear una rama personal de calibración**
- **6. Continuar una rama existente y sincronizarla**
- **7. Crear el entorno virtual e instalar dependencias**
- **8. Habilitar permisos en macOS**
- **9. Preparar la pantalla antes de calibrar**
- **10. Ejecutar una prueba inicial**
- **11. Ajustar coordenadas según la pantalla**
- **12. Ajustar regiones de búsqueda**
- **13. Validar la calibración completa**
- **14. Revisar evidencias y archivos antes de confirmar cambios**
- **15. Confirmar y subir únicamente los archivos necesarios**
- **16. Crear Pull Request y mantener la rama**
- **17. Recomendaciones importantes**

1. Objetivo y regla principal

Cada persona debe calibrar el proyecto en una rama propia, sin trabajar directamente sobre main.

La rama main debe mantenerse limpia, estable y actualizada con el repositorio remoto.

Los cambios de coordenadas deben probarse en la misma resolución, escala, zoom y tamaño de ventana que se usarán normalmente.

2. Requisitos previos

Antes de comenzar, verificar que estén instalados:

- Git
- Python 3
- Terminal, iTerm o PyCharm
- La aplicación remota que se usará durante las pruebas, por ejemplo AnyDesk, RustDesk o Windows App

Verificar versiones:

```
git --version  
python3 --version
```

3. Clonar el proyecto por primera vez

Clonar el repositorio:

```
git clone https://github.com/Franja24/gas-station-bot.git
```

Entrar al proyecto:

```
cd gas-station-bot
```

Verificar el repositorio remoto:

```
git remote -v
```

Nota: este paso solo se realiza la primera vez en cada computadora.

4. Mantener main actualizado antes de trabajar

Antes de crear una rama nueva o comenzar una calibración, actualizar main:

```
git switch main  
git pull --ff-only origin main
```

Si git indica que existen cambios locales, no forzar la actualización. Guardar o revisar esos cambios antes de continuar.

5. Crear una rama personal de calibración

Crear una rama con el nombre de la persona y el equipo o pantalla:

```
git switch -c feature/calibracion-frank-macbook
```

Ejemplos recomendados:

```
feature/calibracion-frank-macbook
```

```
feature/calibracion-ana-monitor1080
```

Verificar la rama actual:

```
git branch --show-current
```

No trabajar directamente sobre main.

6. Continuar una rama existente y sincronizarla

Si la rama ya existe localmente, cambiar a ella:

```
git switch feature/calibracion-frank-macbook
```

Traer los cambios recientes del servidor:

```
git fetch origin
```

Integrar main actualizado en la rama personal:

```
git merge origin/main
```

Si aparecen conflictos, resolverlos y volver a ejecutar las pruebas antes de subir cambios.

7. Crear el entorno virtual e instalar dependencias

Crear el entorno virtual:

```
python3 -m venv venv
```

Activar en Mac/Linux:

```
source venv/bin/activate
```

Activar en Windows:

```
venv\Scripts\activate
```

Instalar dependencias:

```
pip install pyautogui opencv-python pillow mss numpy pynput behave reportlab
```

Nota: la carpeta venv no debe subirse a Git.

8. Habilitar permisos en macOS

En macOS, habilitar permisos para la aplicación desde donde se ejecutará el proyecto:

```
System Settings → Privacy & Security → Accessibility
```

```
System Settings → Privacy & Security → Screen Recording
```

Agregar la aplicación correspondiente:

- PyCharm
- Terminal
- iTerm

Después de cambiar permisos, cerrar y volver a abrir la aplicación.

9. Preparar la pantalla antes de calibrar

Mantener constantes estas condiciones durante la calibración y la ejecución:

- Resolución del monitor
- Escalado de pantalla

Tamaño y posición de la ventana

- AnyDesk / RustDesk / Windows App

Zoom de la aplicación

Nota: si cambia alguna de estas condiciones, las coordenadas pueden dejar de funcionar y será necesario recalibrar.

10. Ejecutar una prueba inicial

Ejecutar desde terminal:

```
python main.py
```

También puede ejecutarse desde PyCharm usando el botón Run.

Antes de editar coordenadas, observar qué clic o búsqueda falla y registrar la posición esperada.

11. Ajustar coordenadas según la pantalla

Las coordenadas se ajustan actualmente en:

```
clicker.py
```

Buscar estructuras como:

```
COORDS = {  
    "continue_button.png": (800, 530),  
}
```

Formato:

```
(x, y)
```

Regla rápida:

- Si el click cae muy a la izquierda → aumentar x
- Si el click cae muy a la derecha → disminuir x
- Si el click cae muy arriba → aumentar y
- Si el click cae muy abajo → disminuir y

Modificar valores poco a poco, ejecutar nuevamente y conservar únicamente coordenadas verificadas.

12. Ajustar regiones de búsqueda

Las regiones de búsqueda también se ajustan actualmente en:

```
clicker.py
```

Ejemplo:

```
REGIONS = {  
    "premium.png": (900, 300, 600, 400)  
}
```

Formato:

```
(x_inicio, y_inicio, ancho, alto)
```

Una región debe incluir completamente el elemento buscado sin abarcar una zona innecesariamente grande.

Después de modificar una región, validar que la imagen se encuentre de forma consistente.

13. Validar la calibración completa

Ejecutar el flujo completo al menos dos veces con la misma configuración de pantalla.

Si existen escenarios BDD, ejecutar:

```
behave
```

Para ejecutar solo un feature:

```
behave features/premium.feature --no-capture
```

Confirmar que los clics, búsquedas y transiciones funcionen sin intervención manual.

14. Revisar evidencias y archivos antes de confirmar cambios

Cada ejecución puede generar una carpeta dentro de:

```
evidencias/
```

También pueden generarse:

```
screenshots/
```

```
execution_report.pdf
```

Revisar el estado del repositorio:

```
git status
```

Nota: no subir evidencias, venv, credenciales, direcciones IP ni archivos temporales a Git.

15. Confirmar y subir únicamente los archivos necesarios

Agregar de forma explícita solo los archivos que se modificaron intencionalmente:

```
git add clicker.py
```

Revisar exactamente lo que se incluirá en el commit:

```
git diff --staged
```

Crear el commit:

```
git commit -m "Calibrate coordinates for Frank MacBook"
```

Subir la rama personal:

```
git push -u origin feature/calibracion-frank-macbook
```

Evitar git add . cuando existan evidencias o archivos locales sin revisar.

16. Crear Pull Request y mantener la rama

Crear un Pull Request desde la rama personal hacia main.

En la descripción, indicar la resolución, escala, aplicación remota y flujo validado.

Antes de agregar cambios nuevos a una rama existente, volver a sincronizarla:

```
git fetch origin
```

```
git merge origin/main
```

Resolver cualquier conflicto y repetir la validación completa.

17. Recomendaciones importantes

No trabajar directamente sobre main.

No forzar pull, merge o push si existen conflictos que no se comprenden.

No asumir que una calibración funcionará en otra pantalla o configuración.

Conservar una rama por persona y contexto de calibración para facilitar revisión y mantenimiento.